

Data Mesh und Data Engineering

(3/4)

Data Engineering

Prof. Dr. Arthur Vetter

Folgende Inhalte werden behandelt

1. OLTP vs. OLAP
- 2. Data Mesh**
3. Data Engineering
4. Data Contracts

Literatur

- Reis, Joe, und Matt Housley. *Fundamentals of Data Engineering: Plan and Build Robust Data Systems*. O'Reilly Media, 2022.
- Konieczny, Bartosz. *Data Engineering Design Patterns: Recipes for Solving the Most Common Data Engineering Problems*. O'Reilly Media, 2025.

Lernziele

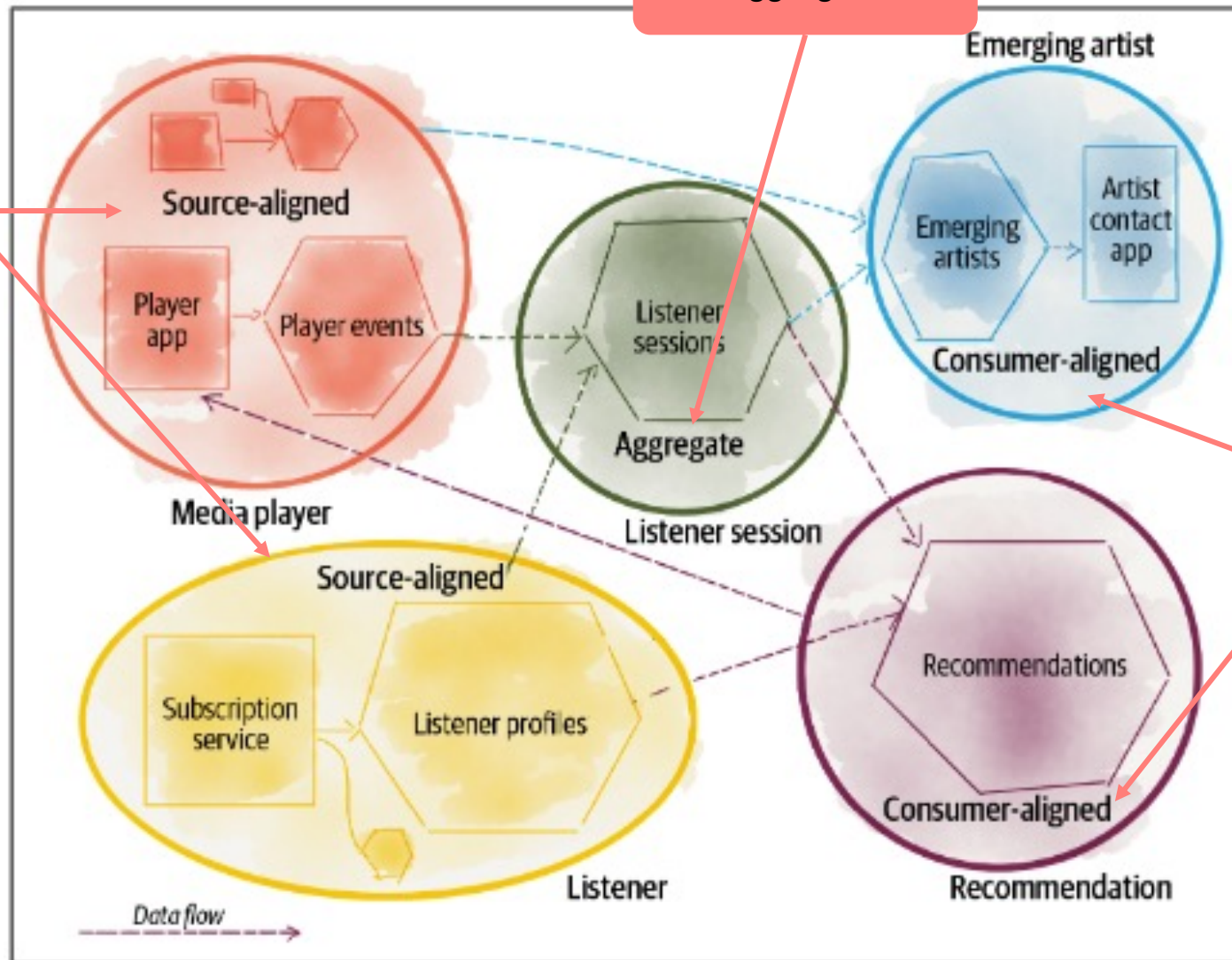
- Verständnis über den Aufbau einer Pipeline erhalten
- Design Pattern kennenlernen
- Streaming vs. Batch Verarbeitung verstehen
- Fähigkeit erlangen, selbst Pipelines bauen zu können

Die drei Typen von Datenprodukten

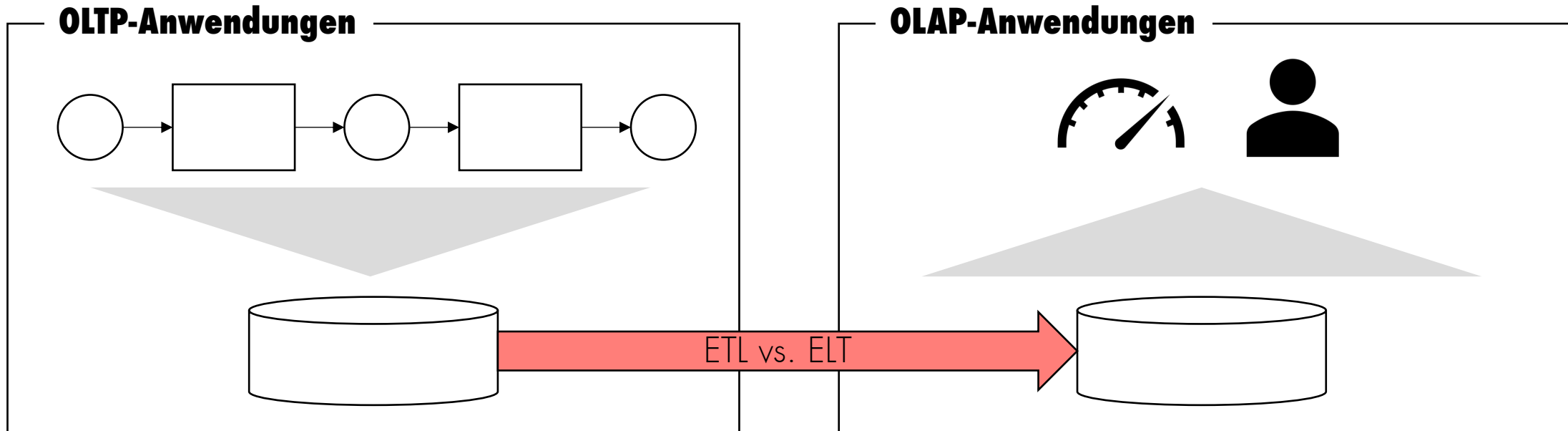
Source-aligned

Aggregate

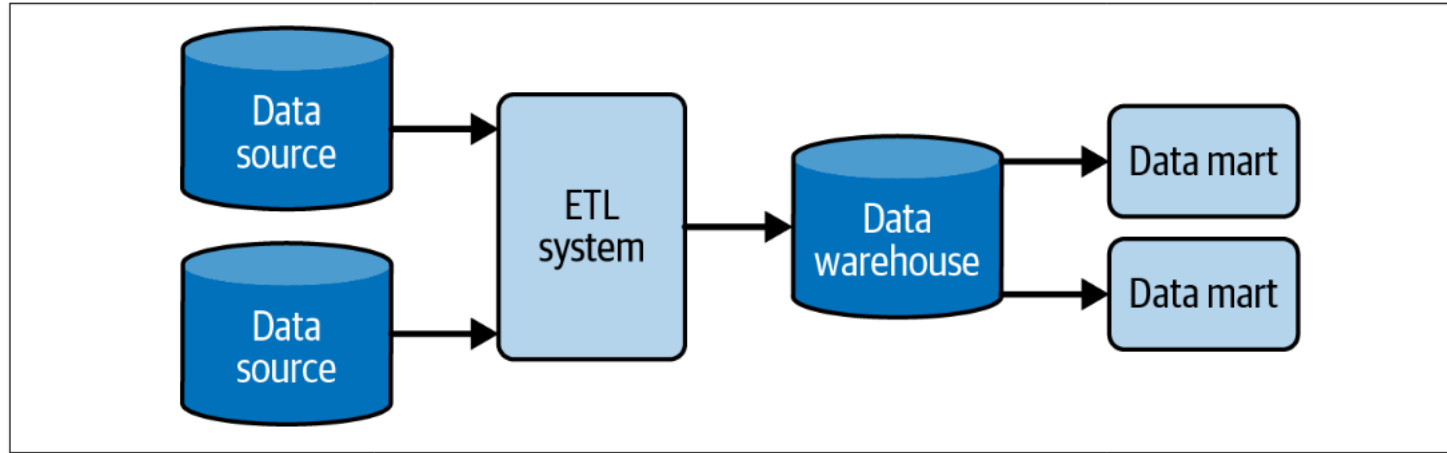
Consumer-Aligned



Source-Aligned Datenprodukte werden mittels ETL bzw. ELT Pipelines in ein OLAP-System geladen



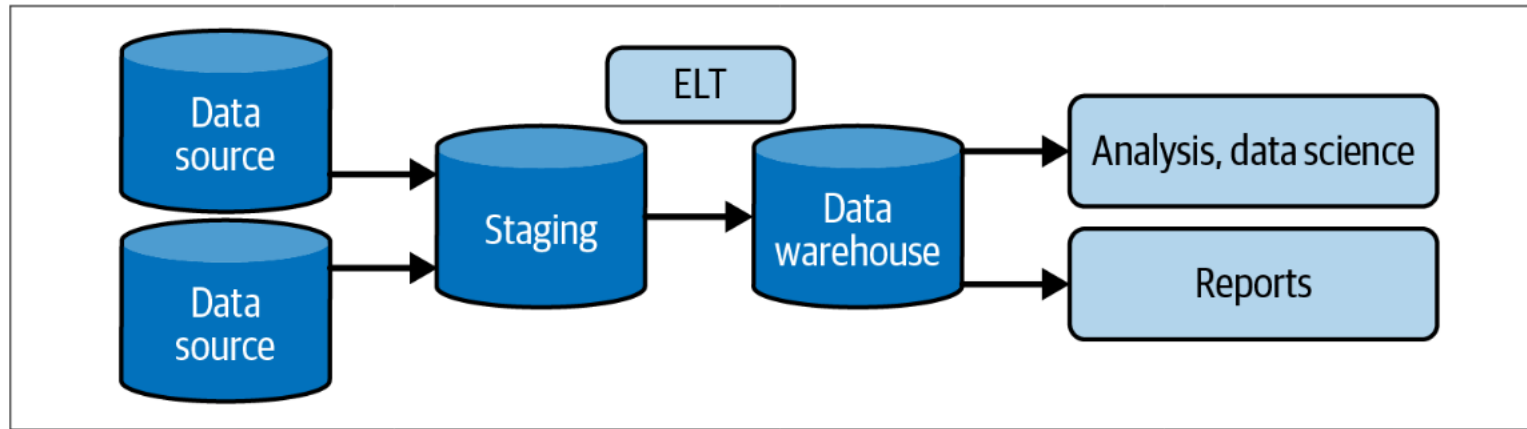
Extract Transform Load



- Extract – Daten aus dem Source-System extrahieren
- Transform – Daten so transformieren, wie man die Daten dann in das OLAP-System laden möchte
- Load – Daten tatsächlich ins OLAP-System laden

▶ Der klassische ursprüngliche Ansatz ein Data Warehouse zu beladen

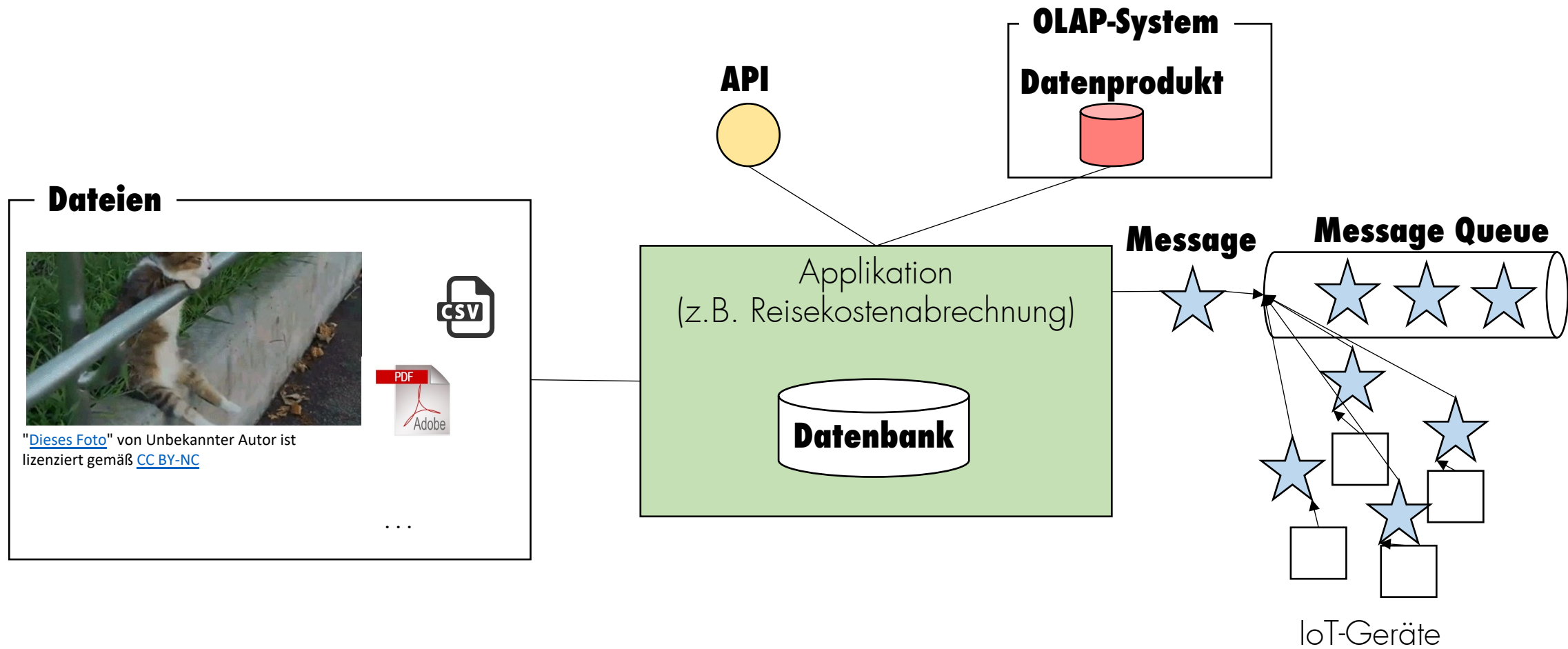
Extract Load Transform



- Extract – Daten aus dem Source-System extrahieren
- Load – Daten roh ins OLAP-System laden. Dadurch können Daten z.B. bei einem Bug in der Transformation erneut direkt gelesen werden, ohne erneut Daten aus dem Source-System extrahieren zu müssen
- Transform – Daten transformiert in Views oder Tabellen bereitstellen oder jeder Use Case transformiert die Daten für sich (transform-on-read ELT)

▶ Hat sich zum Standard im Data LakeHouse entwickelt, da Speicher in der Cloud sehr günstig ist

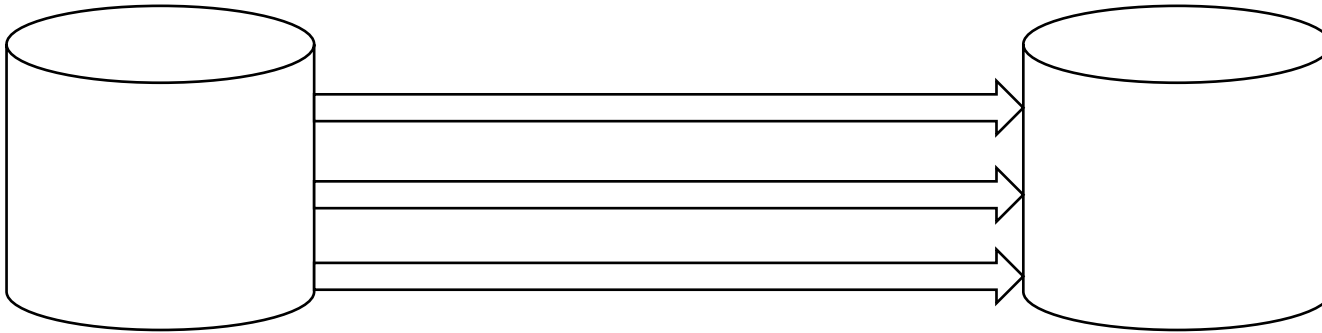
Quellen für Daten



Man unterscheidet generell zwei Arten von Pipelines: Batch vs. Streaming

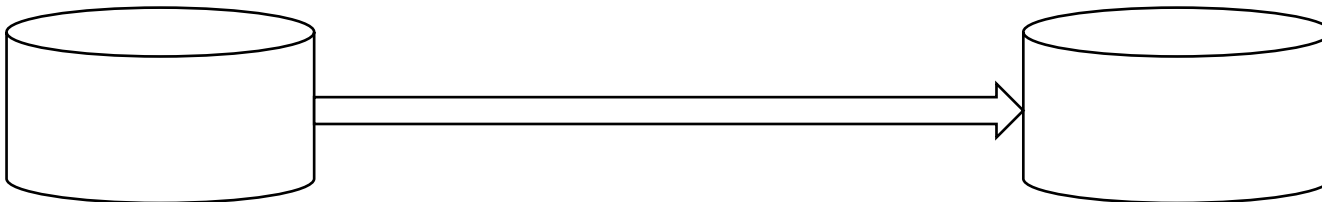
Batch

Daten werden in bestimmten Zyklen von einem System ins andere geladen

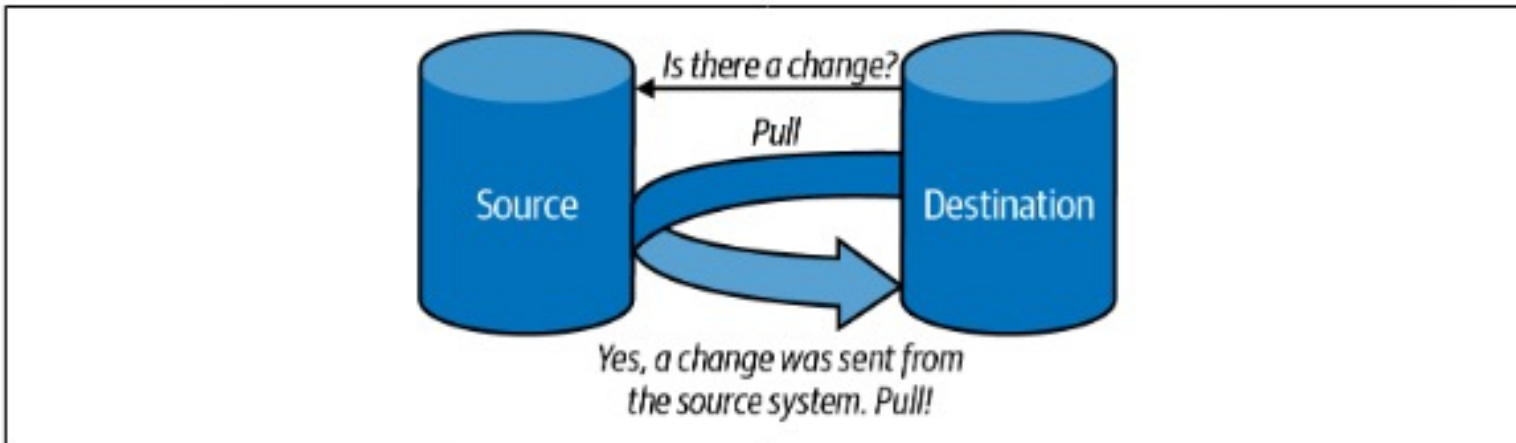
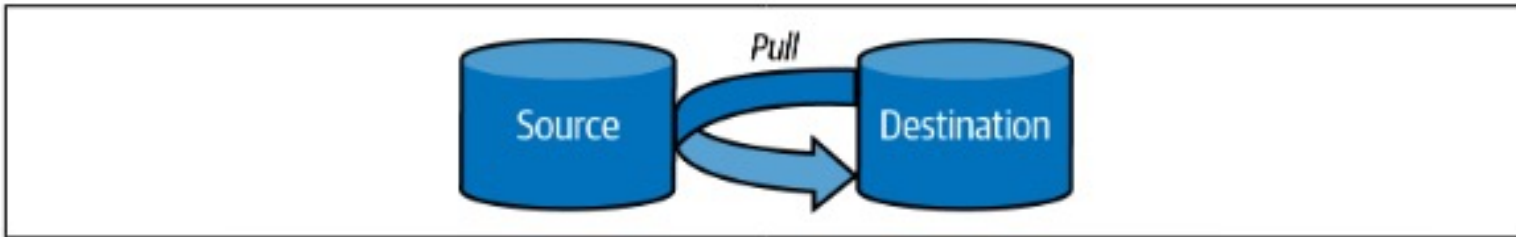
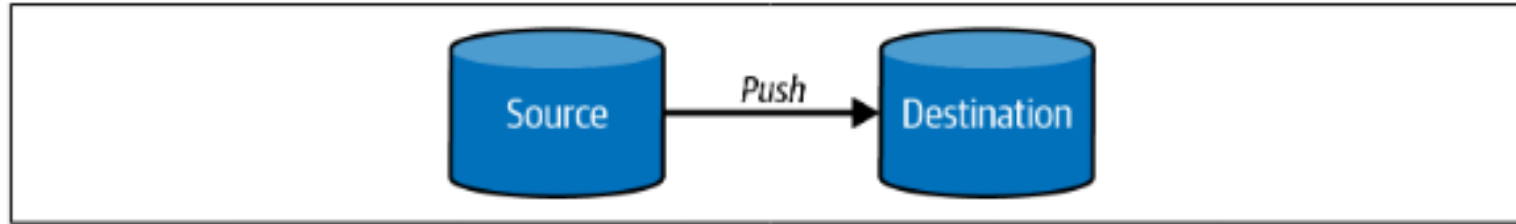


Streaming

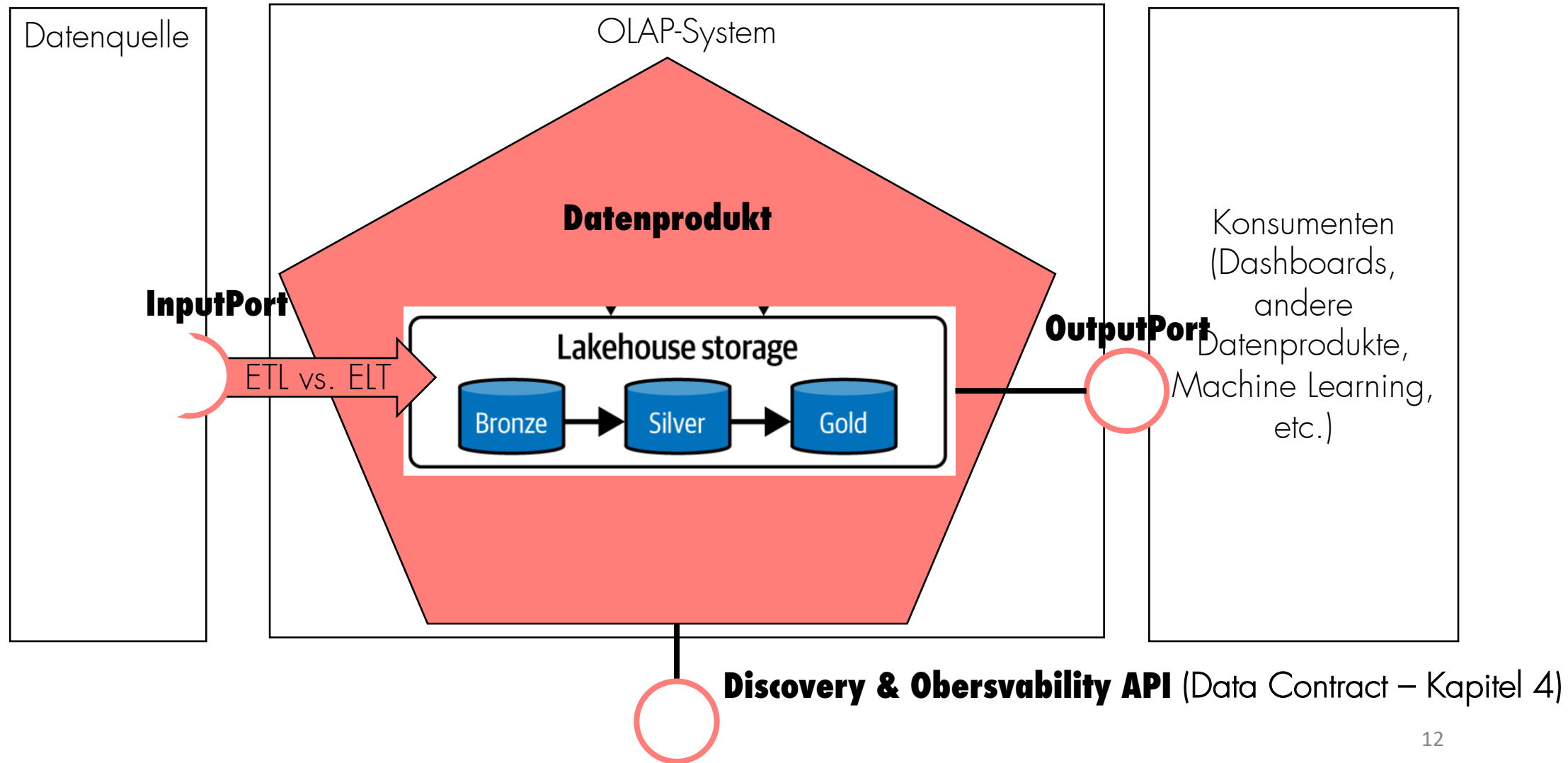
Daten werden kontinuierlich, sobald sie anfallen, in das andere System geladen



Push vs. Pull vs. Poll



Interna eines source aligned Datenprodukts



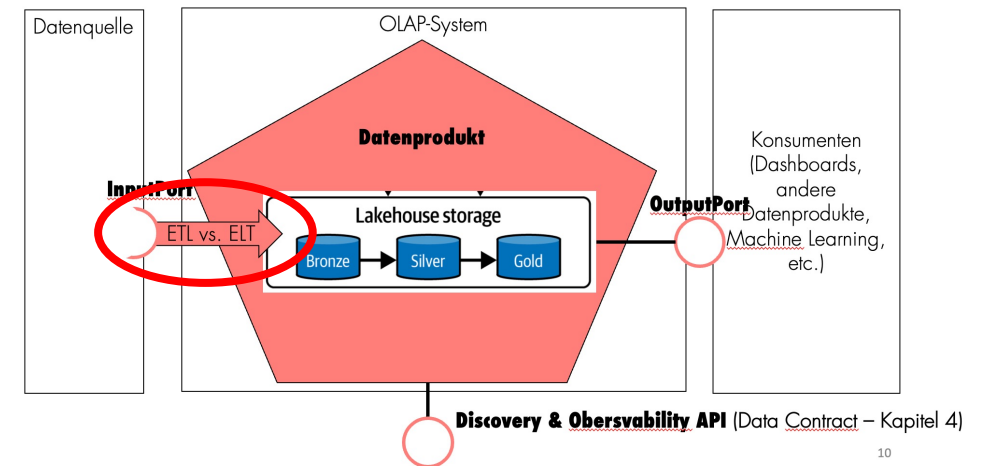
Einige Pattern zum Beladen des Datenprodukts (bzw. des OLAP-Systems)

Full Load

**Incremental Load
(Incremental Loader,
Change Data Capture)**

Batch vs. Streaming

Data Compaction

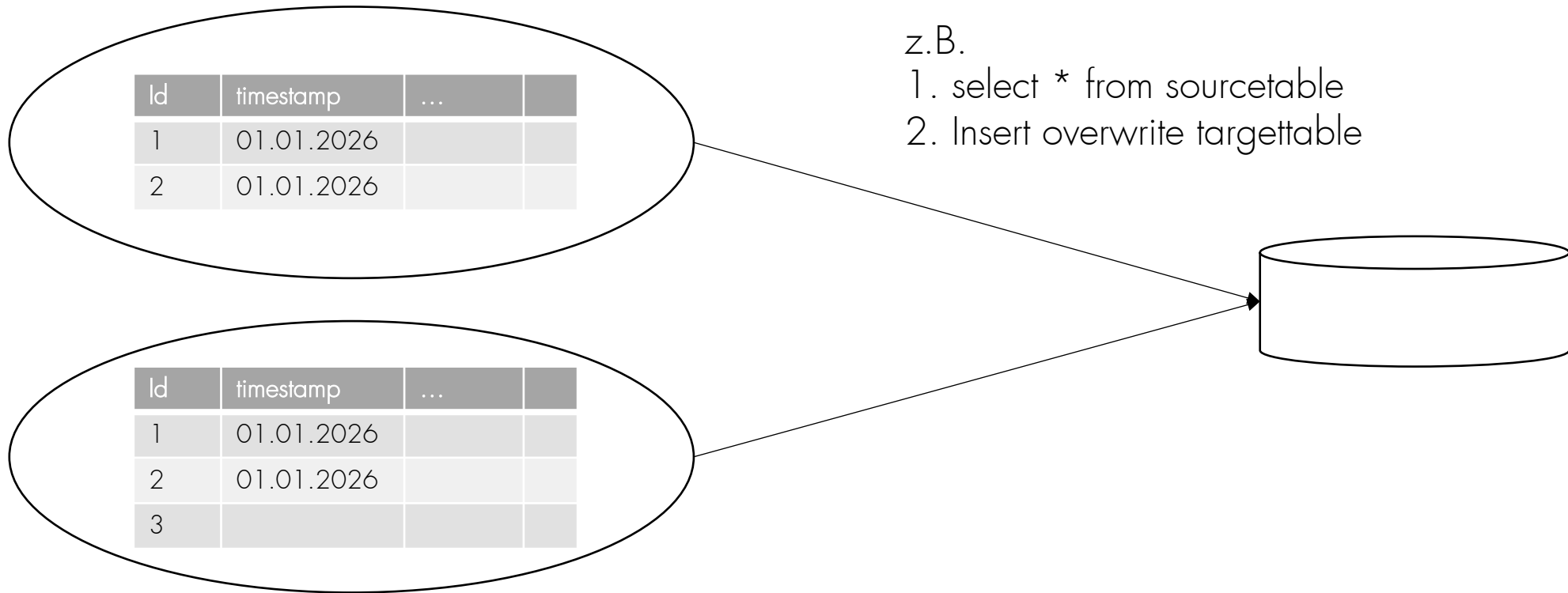


10

Für weitere siehe Konieczny, Bartosz. *Data Engineering Design Patterns: Recipes for Solving the Most Common Data Engineering Problems*. O'Reilly Media, 2025.

Das Full Load Pattern

Es wird immer der gesamte Datensatz aus der Quelle geladen



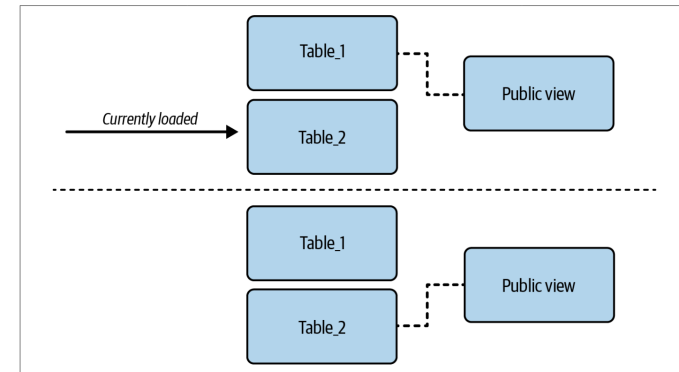
Vorteile und Nachteile des Full Load Patterns

Vorteile

- Einfache Logik
- Immer 1:1 die Daten aus dem Quellsystem

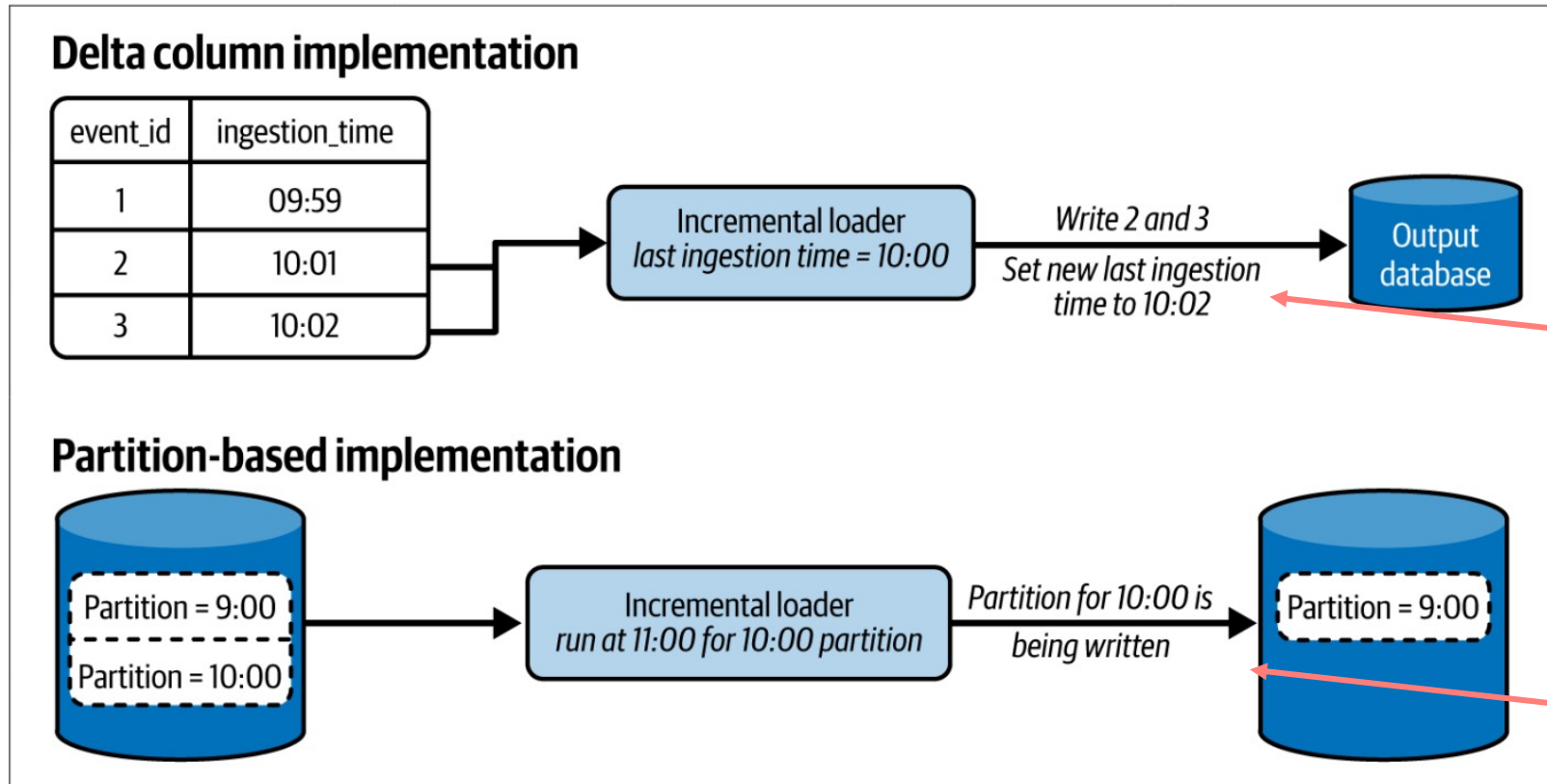
Nachteile

- Bei großen Datenmengen lange Laufzeiten
- Bei Full Loads eher seltenere Ladefrequenzen aufgrund längerer Laufzeiten – bei Bedarf nach Echtzeitdaten oft ungeeignet
- Datenkonsistenz kann beim Lesen eines Konsumenten verloren gehen, kann durch ACID umgangen werden (wenn das OLAP-System es unterstützt) → alternative Lösung über eine Abstraktionsview:



Das Incremental Loader Pattern

Es wird immer das Delta zum letzten Load geladen



Warum sollte man für die Ingest Pipeline eher ingestion_time bzw. processing_time verwenden statt der eigentlichen event_time?

Abfrage auf Basis des letzten Ingests

Abfrage auf Basis letzte Partition ggü. Now

Achtung: Wenn ein Lauf nicht stattfindet, würden Daten fehlen → extran Tools wie Apache Airflow können das handhaben (Backfilling)

Vorteile und Nachteile des Incremental Loader Patterns

Vorteile

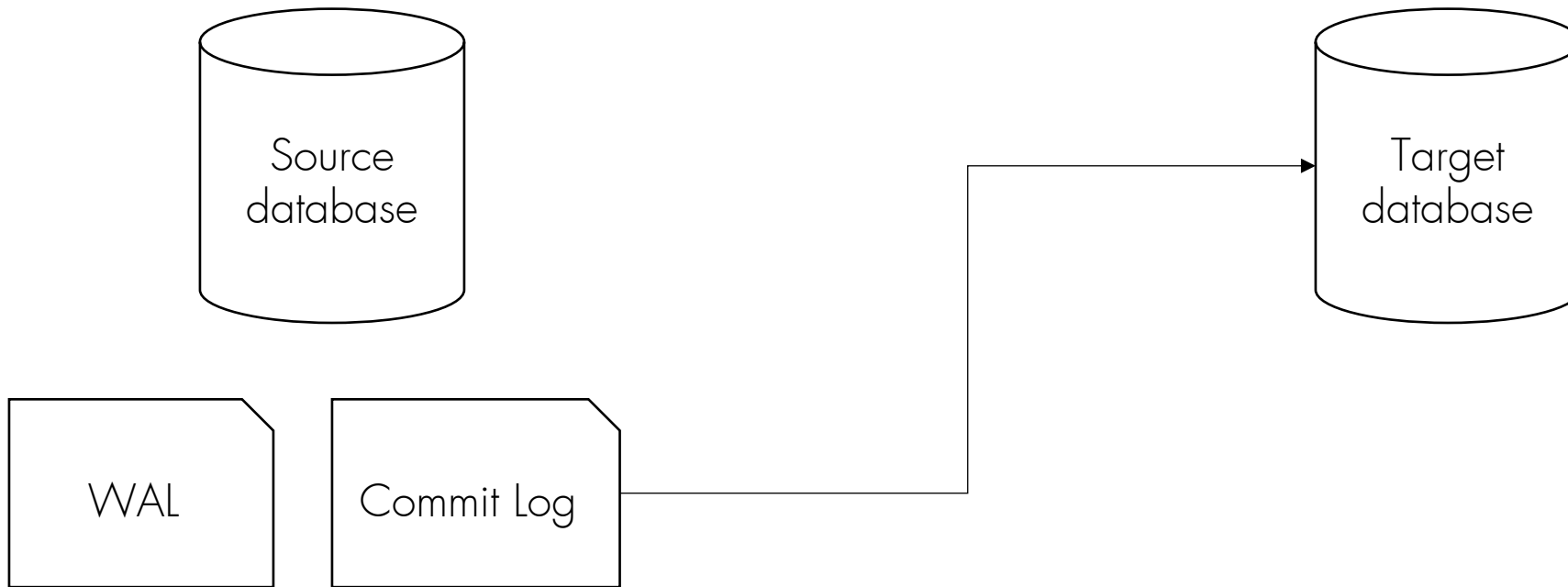
- Die bewegte „Datenmasse“ je Lauf ist geringer, dadurch bessere Performance
- Häufigeres Laden realisierbar

Nachteile

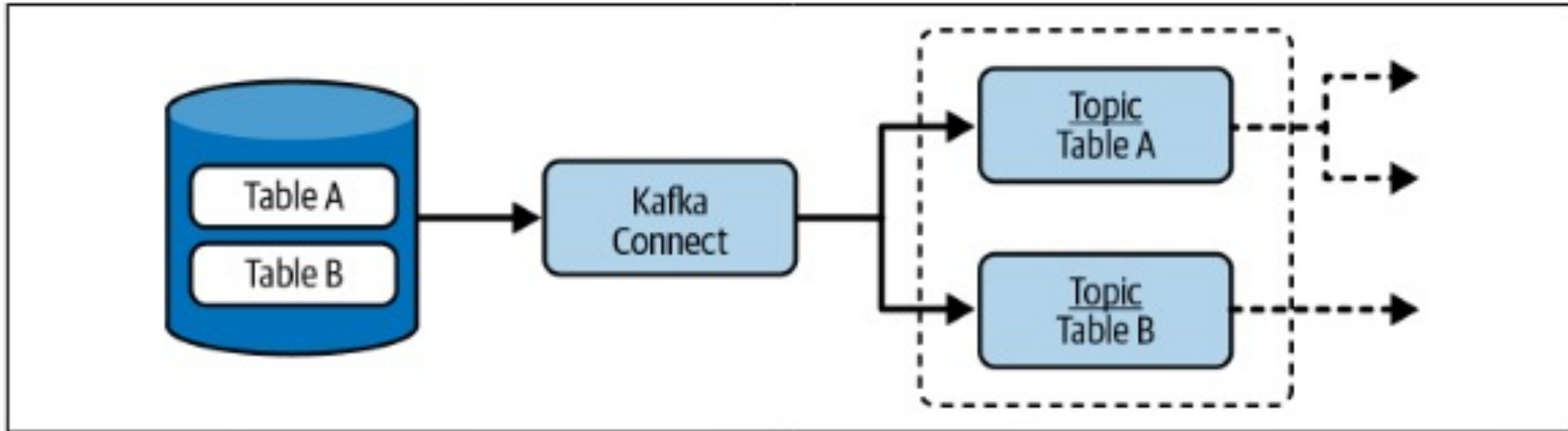
- Komplexerer Umgang mit Deletes im Quellsystem (wie erkennt man eine gelöschte Row? → z.B. durch Soft Deletes: statt Delete Update mit einem „Delete-Flag“)
- Bei Updates auf Rows können Duplikate entstehen → Klären wie das gehandhabt werden soll (ebenfalls die bestehende Row im OLAP-System updaten, oder nur „insert-only“ bzw. „append-only“ und Consumer müssen die Daten bereinigen)

Das Change Data Capture Pattern

Beim Change Data Capture Pattern wird für die Ermittlung der Änderungen das Commit Log ausgewertet und die Änderungen in der Target-Database ebenfalls ausgeführt



Typischer Aufbau mittels Kafka



Vorteile und Nachteile des Change Data Capture Patterns

Vorteile

- Die bewegte „Datenmasse“ je Lauf ist geringer, dadurch bessere Performance
- Häufigeres Laden realisierbar
- Keine Soft-Deletes notwendig

Nachteile

- Zugriff auf Commit – Log notwendig
- Steht nicht immer für alle Datentypen zur Verfügung
- Pipeline und Ingest in Zieltabelle muss schnell genug sein, sonst können Änderungen aus dem Log entfernt werden und dann verpasst werden
- Deshalb oft Lösung mit einem Broker wie Kafka
→ relativ aufwändig

Übung 1: Erstellen Sie in Nifi einen Flow und implementieren Sie einmal das Full Load Pattern und einmal das Incremental Loader Pattern

- Erstellen Sie in Impala zunächst für sich die zwei Tabellen im Skript „Nifi create tables“ aus Moodle – Nutzen Sie als Präfix ihren WI-User
 - Erstellen Sie im Data Flow Service auf der Cloudera-Plattform ein neues Projekt
 - Kopieren Sie den Flow „JDBC to JDBC“ aus dem Catalog in ihr Projekt (Create New Draft)
 - Passen Sie den Flow so an, dass ihre Tabellen verwendet werden und lassen Sie den Flow mehrmals laufen → Was fällt Ihnen auf?
-
1. Passen Sie den Flow so an, dass das Full Load Pattern verwendet wird und testen Sie ihn
 2. Passen Sie den Flow so an, dass das Incremental Loader Pattern verwendet wird und testen Sie ihn

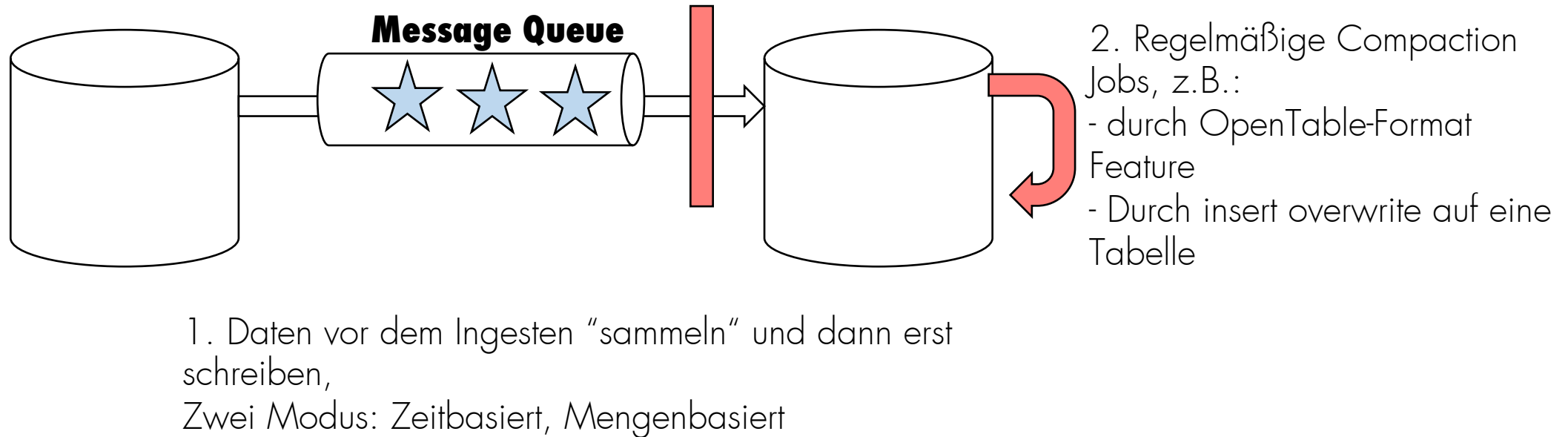
Das Compaction Pattern

- Insbesondere bei Streaming Pipelines können in kurzer Zeit sehr viele kleine Files entstehen
- Viele kleine Files verlangsamen die Lese-Performance, weil die I/O-Last sehr hoch wird



Das Compaction Pattern

- Dies kann auf zwei Wegen gelöst werden (auch in Kombination)



Übung2: Erstellen Sie in Nifi einen Flow und implementieren Sie einmal das Full Load Pattern und einmal das Incremental Loader Pattern

- Nutzen Sie ihren incremental Flow und ergänzen Sie ihn um einen "MergeRecord"-Prozessor, der die zu übertragenden Daten compacted
- Testen Sie ihren Flow, indem sie in ihre source-Tabelle neue Einträge hinzufügen
- Um das weiter zu automatisieren, können Sie ihren Nifi-Flow erweitern, dass z.B. jede Minute ein neues Flow-File erstellt wird, welches das Lesen aus der source-Tabelle triggert (GenerateFlowFile – Prozessor)

Einige Pattern zur Erreichung von Idempotenz

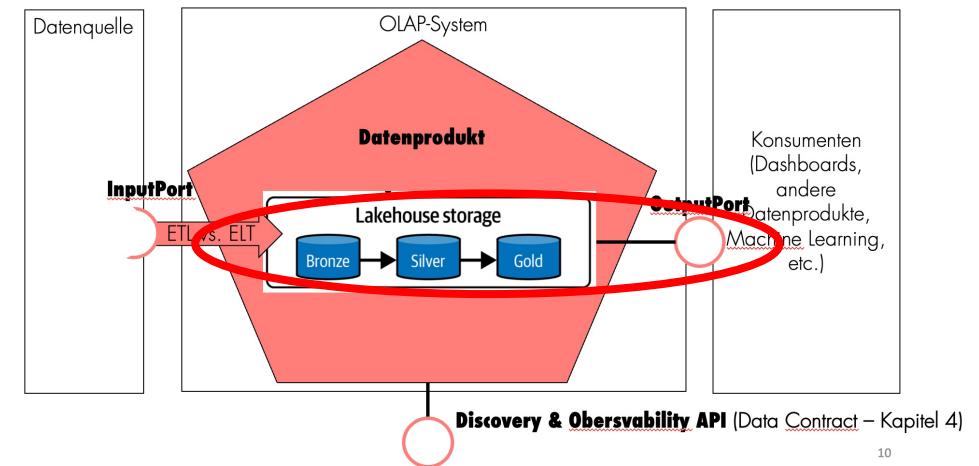
Idempotenz bedeutet, dass eine Operation mehrmals hintereinander ausgeführt werden kann, ohne dass sich das Ergebnis nach der ersten Anwendung ändert.

Truncate/Drop

Data Overwrite

Upsert

Proxy-View

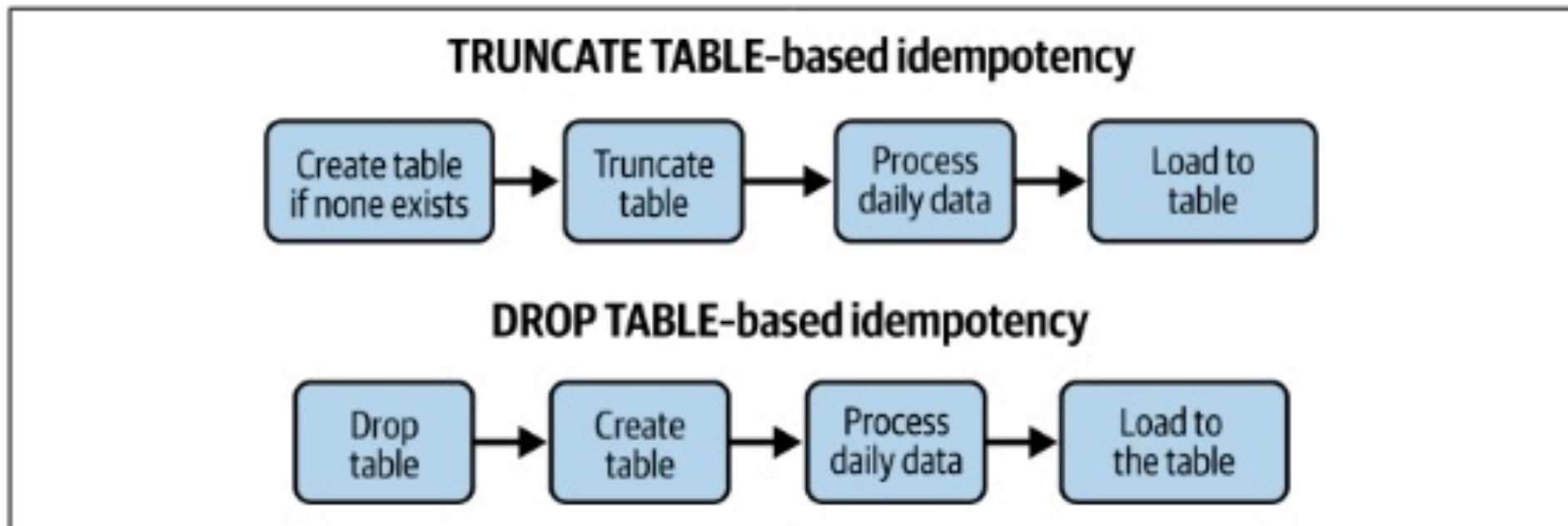


10

Für weitere siehe Konieczny, Bartosz. *Data Engineering Design Patterns: Recipes for Solving the Most Common Data Engineering Problems*. O'Reilly Media, 2025.

Truncate/Drop - Pattern

- Idee: Bevor neue Daten hinzugefügt werden, wird die alte Tabelle gedroppt/truncated und danach werden die neuen Daten eingefügt (insert/load). In der Regel in Kombination mit Partitionen verwendet
- In Lakehouses erfolgt ein Drop auf externe Tabellen rein auf Metadatenebene (sehr günstige Operation)



Vorteile und Nachteile des Truncate/Drop - Patterns

Vorteile

- Relativ schnelle Operation (löschen ist schneller als updaten)
- Load-Command operiert rein auf File-Ebene und ist sehr schnell (verschiebt Dateien) anstatt einzeln einzufügen wie bei insert

Nachteile

- Besser geeignet für kleine Tabellen oder gut partitionierte Tabellen
- Wenn keine Transaktionen unterstützt werden, können User kurzzeitig nicht auf Daten zugreifen (eine mögliche Lösung siehe Data Full Load Pattern)
- Bei Neuanlage vieler neuer Partitionen, ohne Entfernung der alten können schnell Limits der Lösungen erreicht werden

Übung3: Erstellen Sie eine externe Tabelle, dropen Sie diese und erstellen Sie erneut eine neue Tabelle

- Laden Sie die in Moodle bereitgestellte Datei „bevoelkerungszahlen“ in Ihren persönlichen Ordner in S3 unter folgendem Pfad:
data/user/csso_... (falls ihr User nicht existiert erstellen Sie einen Ordner über Hue)
- Merken Sie sich den Pfad und erstellen Sie nun eine externe Tabelle für diese Daten, siehe:
https://impala.apache.org/docs/build/html/topics/impala_create_table.html#create_table
- Als Orientierung können Sie auch ein „show create table project_bevoelkerungszahlen“ ausführen, um zu sehen, wie ich diese Tabelle angelegt habe
- Wenn Sie die Tabelle erzeugt haben finden Sie heraus wie viele Einwohner Stuttgart zuletzt hatte
- Dropen Sie nun die Tabelle und prüfen Sie, ob ihre Daten auf File-Ebene (Pfad von oben) noch bestehen
- Erstellen Sie die Tabelle erneut, aber nur noch mit den Spalten für 2024. Finden Sie erneut heraus wie viele Einwohner Stuttgart hat.
- Kopieren Sie die Datei „bevoelkerungszahlen“ und benennen Sie sie um zu „bv_v2“. Laden Sie diese Datei ebenfalls in denselben S3-Ordner hoch.
- Rufen Sie erneut die Tabelle auf. Welche Ergebnisse sehen Sie nun für Stuttgart?
- Führen Sie folgenden Befehl aus und lesen Sie danach erneut die Ergebnisse aus der Tabelle: “invalidate metadata <ihre_tabelle>

Data Overwrite - Pattern

- Mit insert overwrite werden **alle** bestehenden Daten in einer Tabelle überschrieben
- Vorteile: kein zusätzlicher Orchestrierungsaufwand
- Nachteile: Für große Datenmengen langsam

Upsert/Merge - Pattern

UPSERT Statement (Impala 2.8 or higher only)

Acts as a combination of the `INSERT` and `UPDATE` statements. For each row processed by the `UPSERT` statement:

- If another row already exists with the same set of primary key values, the other columns are updated to match the values from the row being "UPSERted".
- If there is not any row with the same set of primary key values, the row is created, the same as if the `INSERT` statement was used.

This statement only works for Impala tables that use the Kudu storage engine.

- Statt wie beim Insert Overwrite werden bestehende Zeilen upgedatet und neue hinzugefügt
- Es muss eine unique-Spalte geben, um die upzudatende Row zu erkennen

Vorteile und Nachteile des Upsert - Patterns

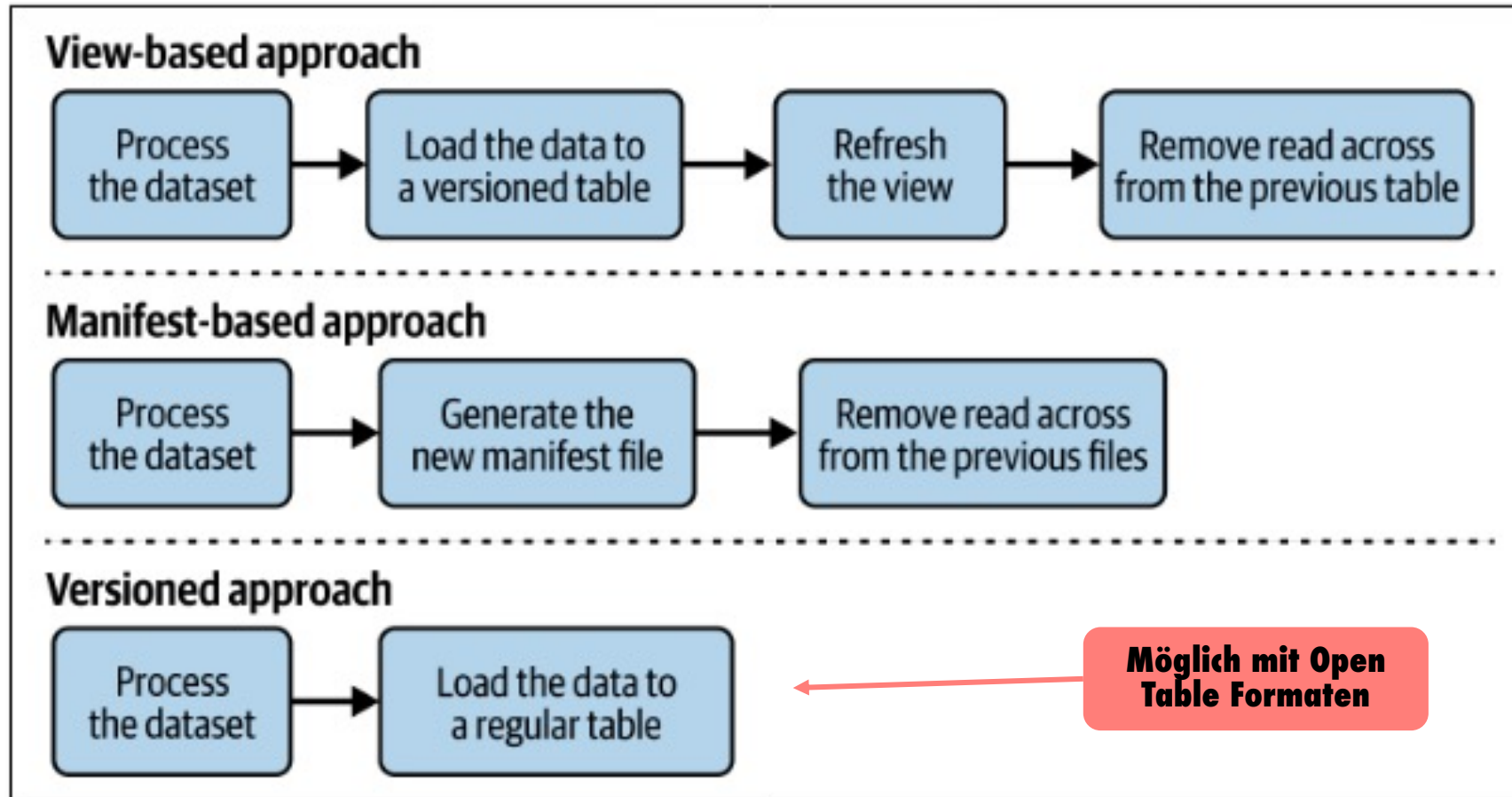
Vorteile

- Bei kleinen Datensätzen müssen nicht alle Daten gelöscht werden, sondern nur ausgewählt

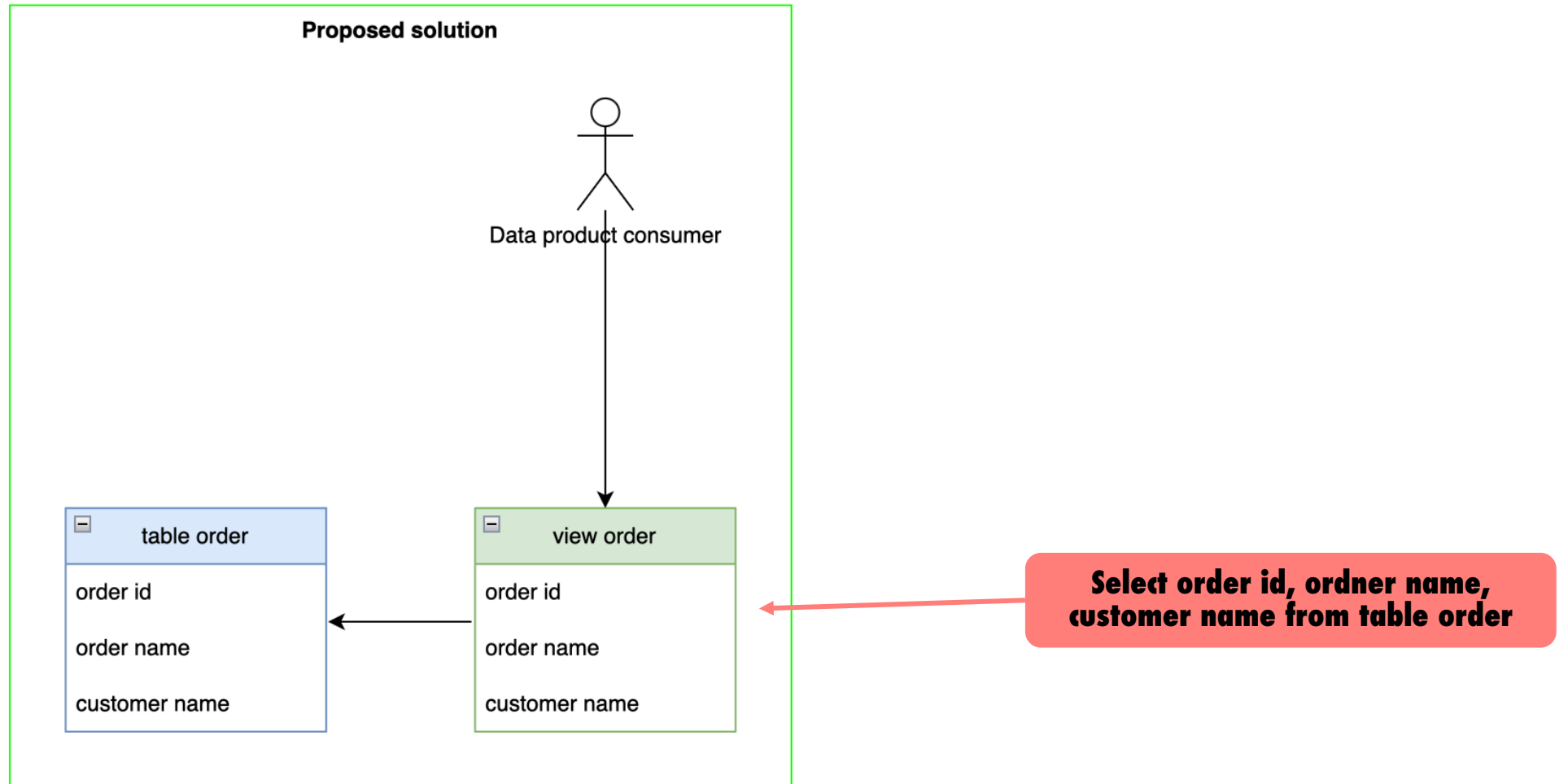
Nachteile

- Wenn die Mehrzahl der Rows upgedatet werden muss, dann ist das insert overwrite bzw. Drop/truncate Pattern effizienter
- Löschen funktioniert damit nicht → Soft Deletes können verwendet werden

Proxy View - Pattern



Proxy View - Pattern



Vorteile und Nachteile des Proxy - Patterns

Vorteile

- Man hat immer ein stabiles Interface ggü. Konsumenten
- Duplikate können leicht durch ein distinct herausgefiltert werden

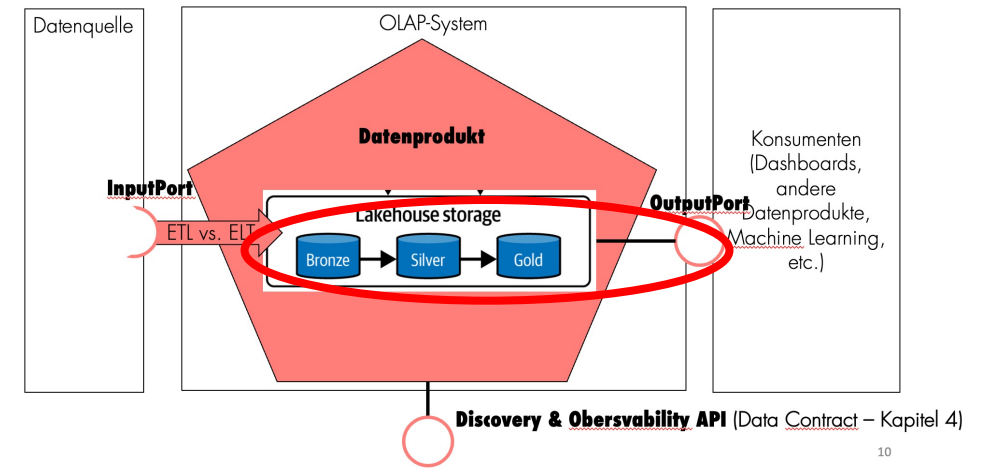
Nachteile

- Bei Nutzung von Distinct → Verschlechterung der Performance → nur als temporäre Lösung nutzen
- Bei Einsatz mehrerer Tabellen zusätzlicher Aufwand zur Wartung der View

Einige Pattern zur Organisation von Daten

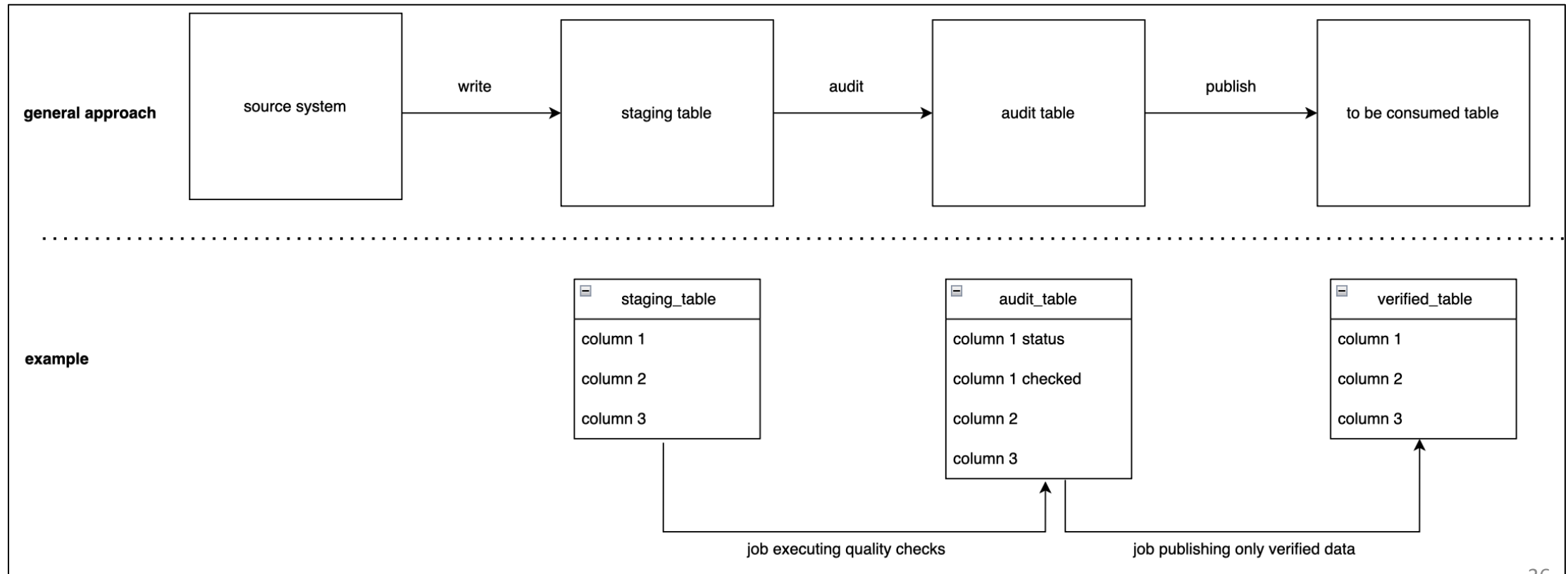
Write Audit Publish

Medallion Architecture



Write Audit Publish (WAP) - Pattern

Bevor Daten Konsumenten über den OutputPort konsumierbar bereitgestellt werden, müssen diese zunächst einen Audit durchlaufen (Statistiken und fachliche Checks)



Vorteile und Nachteile des WAP - Patterns

Vorteile

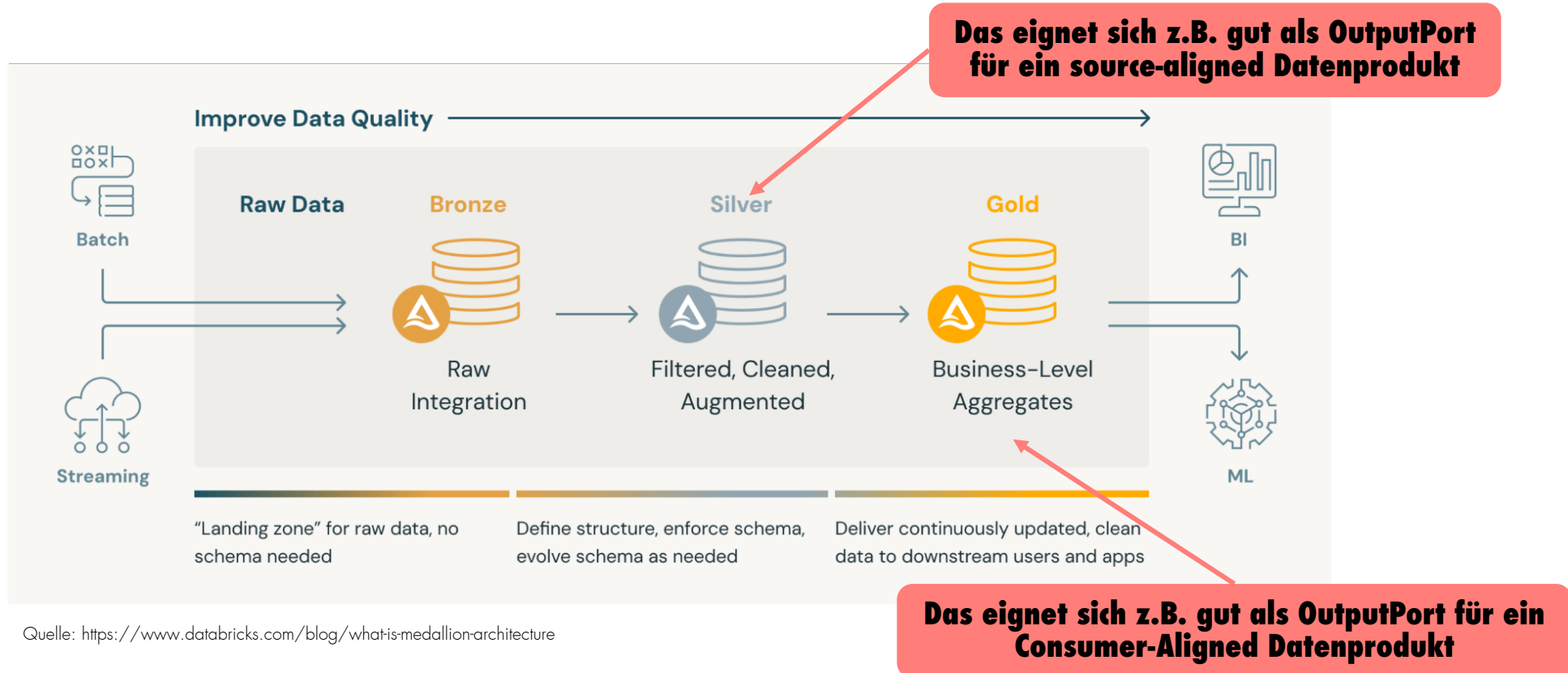
- Falsche Daten werden nicht für Analysen verwendet
- Vertrauen in die Daten und das Team wächst

Nachteile

- Zusätzlicher Entwicklungs- und Betriebsaufwand (auch Compute- und Speicherkosten)
- Audits müssen regelmäßig kontrolliert werden
- Es können trotzdem fehlerhafte Daten konsumiert werden, wenn diese noch nicht durch die Audit-Regeln erkannt werden
- Es könnten auch fälschlicherweise Daten herausgefiltert werden

Medallion Architecture - Pattern

Design-Pattern zur Organisation von Daten in einem Datenprodukt



Vorteile und Nachteile des Medallion Architecture - Patterns

Vorteile

- Klare Strukturierung der Pipeline
- Einzelne Schritte in der Pipeline können unabhängig voneinander ausgeführt werden

Nachteile

- Zusätzlicher Entwicklungs- und Betriebsaufwand (auch Compute- und Speicherkosten)

Zum Entwickeln komplexerer Pipelines, insbesondere mit viel Transformationslogik eignet sich Spark besser als Nifi

- <https://spark.apache.org>
- Spark ist eine Engine, die verteilt arbeitet und mit sehr großen Datenmengen arbeiten kann
- Entwickelt an der University of California, Berkeley:
Zaharia, Matei, Mosharaf Chowdhury, Tathagata Das, u. a. „Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing“. *Proceedings of the 9th USENIX conference on Networked Systems Design and Implementation* (USA), NSDI'12, 25. April 2012, 2.
- Kommerzialisiert durch das Unternehmen Databricks

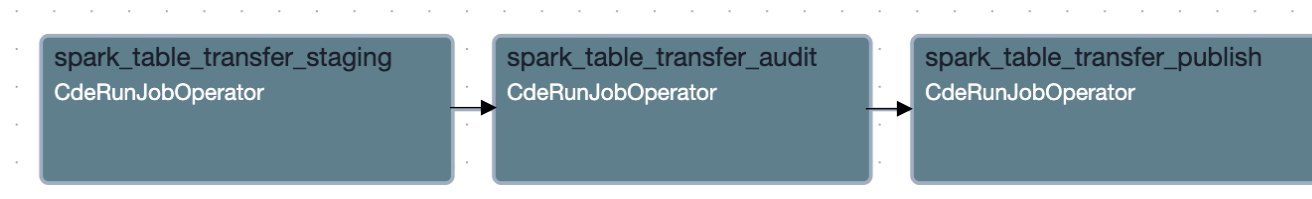


Spark ist in Java geschrieben, bietet aber auch APIs für andere Sprachen

- Java, Scala, Python (PySpark), R (SparkR)
- Unterstützung von Data Frames als auch direkt SQL
- Code wird immer in einer JVM ausgeführt
- Lazy evaluation (erst wenn Methoden aufgerufen werden, die sofort ausgeführt werden müssen, z.B. `.show()`, `.collect()`, DDL-Statements werden diese tatsächlich ausgeführt)
- Gut geeignet für komplexe Transformationen, da auf Code-basiert gearbeitet werden kann

Um Pipelines zu schedulen gibt es eigene Tools

- Einfache Lösung: cron-Jobs (auf Linux/Unix-Systeme → defacto Standard im Data Engineering Umfeld)
- Komplexere Lösungen wie Apache Airflow: <https://airflow.apache.org> basieren häufig einem DAG (Directed Acyclic Graph)
 - Directed bedeutet gerichtet, d.h. jede Verbindung hat eine Richtung, z.B. von einem Knoten zu einem anderen
 - Acyclic bedeutet, dass es keine Schleifen geben kann



Übung4: Analysieren Sie den DAG „DAG_incremental“ im Data Engineering Service

- Welche Spark-Jobs werden im DAG ausgeführt?
- Was machen die einzelnen Spark-Jobs? Tipp: Schauen Sie sich die Skripte in Moodle an, oder unter „Resources/table2table_job)
- Erstellen Sie selbst einen DAG, der Daten von einer Quell-Tabelle in eine Ziel-Tabelle schreibt. Anschließend soll in einer Metrik-Tabelle je Lauf die Row-Anzahl der zuletzt geschriebenen Rows geschrieben werden.
- Sie können sich an dem Spark-Skripten von oben orientieren.

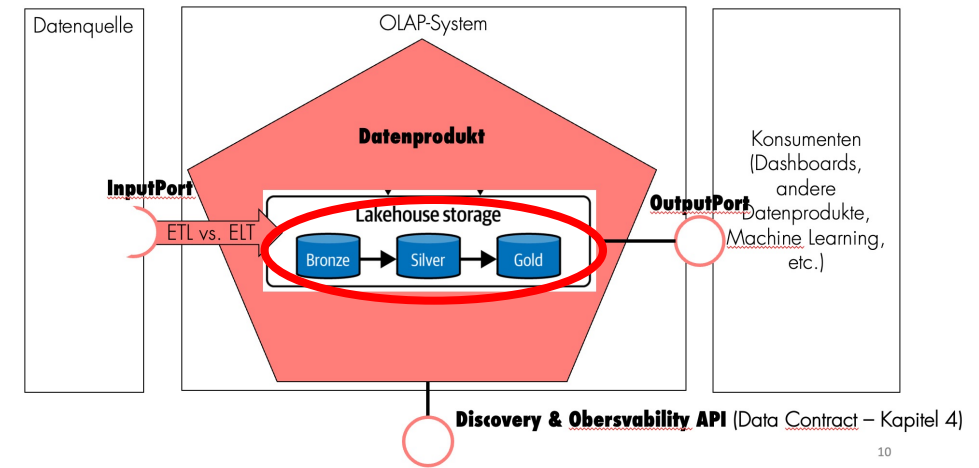
Eine Best Practices, um eine hohe Performance bei Datenabfragen für ein Datenprodukt zu erhalten

Data Compaction

Defragmentation

Partitionierung

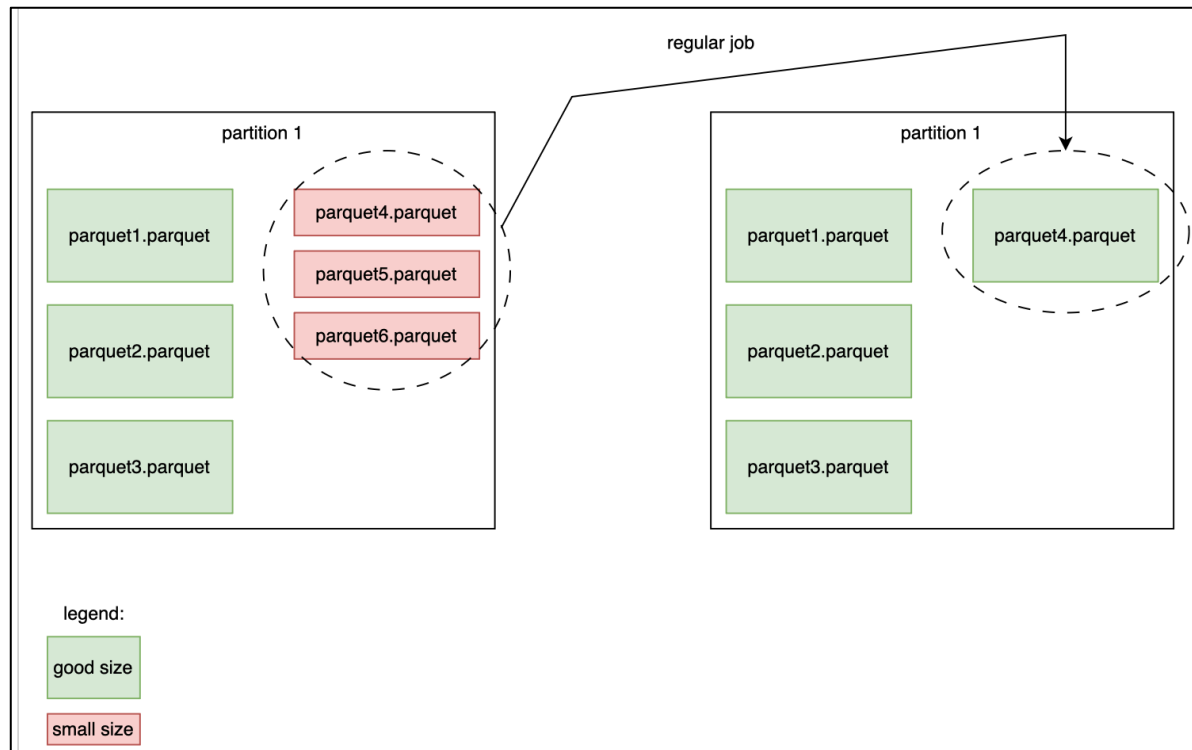
Sorting



Data Compaction

Stellen Sie sich eine Pipeline vor, die jeden Tag eine neue Datei mit 10 MB zu einer Tabelle hinzufügt. Nach einem Jahr würde die Tabelle 365 Dateien enthalten, von denen jede nur 10 MB groß ist.

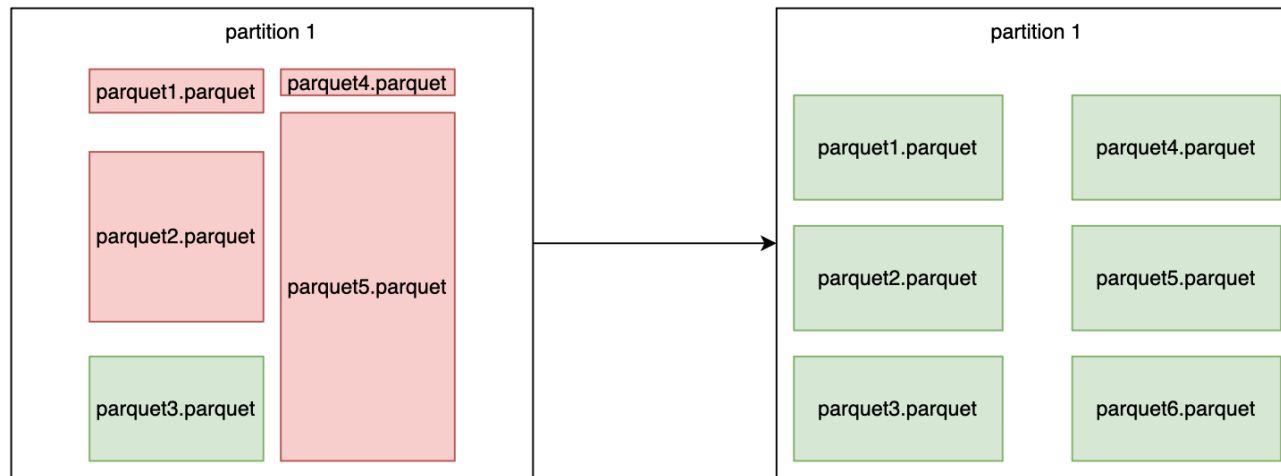
Ein besserer Ansatz ist es, weniger, dafür größere Dateien zu haben – zum Beispiel etwa 14 Dateien mit jeweils 256 MB, die die Daten eines gesamten Jahres enthalten.



Was ist der Trade Off bei der Auswahl der Dateigröße?

Defragmentation

Wenn Dateien nicht ähnlich groß sind, verlangsamt sich auch die Leistung, da gleichzeitige Anfragen von einer Engine nicht effizient verarbeitet werden können. Im Allgemeinen ist es gut, ähnliche Dateigrößen zu haben.



legend:

good size

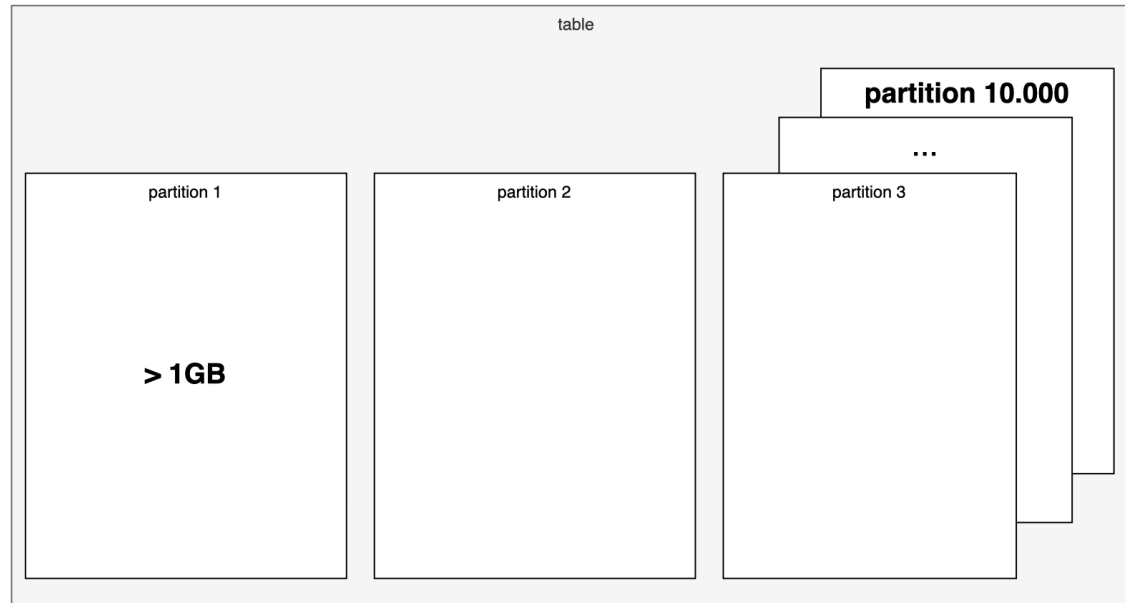
random size

Kann z.B. leicht passieren bei Initialbeladung vs. Delta-Beladung

Partitionierung

Tabellen eines Datenprodukts können sehr groß werden, was zu einer langsamen Leseperformance und einem hohen Wartungsaufwand führt – beispielsweise wenn im Fehlerfall eine gesamte Tabelle neu geladen werden muss. Die Partitionierung von Daten kann die Leseperformance verbessern und gleichzeitig die Verwaltung eines Datenprodukts effizienter gestalten.

Allerdings können zu viele Partitionen oder zu kleine Partitionen die Performance und Verwaltung sogar verschlechtern.



Übung5: Erstellen Sie eine partitionierte Tabelle

- Erstellen Sie eine Iceberg-Tabelle mit folgenden Spalten: id, datum
- Partitionieren Sie die Tabelle nach dem Jahr in dem ein Record eingefügt wurde (im create table Statement „Partitioned by SPEC (year (datum))“ verwenden)
- Fügen Sie einige Datensätze hinzu, z.B:

```
INSERT INTO my_table PARTITION (year)
VALUES
```

```
(1, CAST('2025-01-15' AS DATE)),
(2, CAST('2025-06-20' AS DATE)),
(3, CAST('2026-03-10' AS DATE));
```

- Prüfen Sie in S3 wie die Ordnerstruktur aussieht und die Files, die in den Ordner erstellt wurden (Den Speicherort können Sie mit dem Befehl „Describe Formatted <tablename>“ ausfindig machen)

Sortierung

Sortierung ist ein sehr mächtiges Hilfsmittel, um die Performance bei Datenabfragen zu erhöhen. Es gibt verschiedene Sortiermöglichkeiten (z-order, aufsteigend, etc.)

```
CREATE [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]table_name
  (col_name data_type
    [constraint_specification]
    [COMMENT 'col_comment']
    [, ...]
  )
  [PARTITIONED BY (col_name data_type [COMMENT 'col_comment'], ...)]
  [SORT BY ([column [, column ...]])]
  [COMMENT 'table_comment']
  [ROW FORMAT row_format]
  [WITH SERDEPROPERTIES ('key1'='value1', 'key2'='value2', ...)]
  [STORED AS file_format]
  [LOCATION 'hdfs_path']
  [CACHED IN 'pool_name' [WITH REPLICATION = integer] | UNCACHED]
  [TBLPROPERTIES ('key1'='value1', 'key2'='value2', ...)]
```

Was ist der Trade Off beim Sortieren?
Was ist schnell und was langsam?

Zusammenfassung

- Gängige Pipeline-Pattern sind ETL bzw. ELT
- Es wird unterschieden zwischen Streaming und Batch-Verarbeitung
- Es gibt verschiedene Pattern zum Beladen von Tabellen aus Quellen (full load vs. Incremental load)
- Pattern zum Organisieren der Daten in einem Datenprodukt sind die Medallion Architektur oder das WAP-Pattern
- Um eine gute Performance zu erreichen sind auch Optimierungen auf Storage-Ebene zu berücksichtigen
- Nifi ist eine gängige Plattform für Streaming und gut geeignet für einfache Transformationen
- Spark ist der defacto-Standard und eignet sich auch für komplexe Transformationen